

WEF1VO Web Fundamentals

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

WS 2025/26

Vorlesung 6

Web Accessibility

6.1 Allgemeines

Web Accessibility (engl. wörtlich „Zugänglichkeit“), auch oft barrierefreies Web (oder fälschlich barrierefreies Internet) oder kurz A11Y (für Accessibility – zwischen A und Y sind 11 Buchstaben) genannt, bezeichnet die Praxis, Websites so zu gestalten, dass sie für alle Personen – unabhängig von Behinderungen oder sonstigen Einschränkungen – verwendbar sind.

Tim Berners-Lee, Vorsitzender des W3C und Erfinder des World Wide Web, drückt dies wie folgt aus [8]:

The power of the Web is in its universality. Access by everyone regardless of disability is an essential aspect.

Genauer ausgedrückt bedeutet dies, dass Personen mit Behinderungen das Web wahrnehmen, verstehen, navigieren, mit ihm interagieren und zum Web etwas beitragen können sollen. Dabei werden alle Behinderungen gleichermaßen berücksichtigt:

- Sehbehinderungen (z. B. Blindheit, Farbenblindheit, geringe Sehkraft),
- Hörbehinderungen (z. B. Gehörlosigkeit, Schwerhörigkeit),
- physische Behinderungen (z. B. Amputationen, Arthritis, Muskeldystrophie),
- kognitive und neurologische Behinderungen (z. B. Aufmerksamkeitsdefizit- oder Hyperaktivitätsstörung, Legasthenie, Epilepsie, Gedächtnisstörung),
- Sprachbehinderungen (z. B. Stummheit, Stottern, Sprechapraxie).

Darüber hinaus nimmt Web Accessibility aber auch auf ältere Nutzer*innen Rücksicht, die das Web oft aufgrund reduzierter motorischer Fähigkeiten nur mehr eingeschränkt nutzen können. Ebenso werden Personen mit temporären Einschränkungen, wie z. B. einem gebrochenen Arm unterstützt oder auch Personen, die von technologischen Limitierungen wie etwa geringer Bandbreite oder veralteten Geräten betroffen sind. Dies fällt explizit nicht unter den Begriff „Behinderung“, weshalb oft auch der generellere Ausdruck „Einschränkung“ zum Einsatz kommt.

Das W3C stellt auf seiner Seite Szenarios von Web User*innen mit Einschränkungen¹ vor, die als Beispiele dafür dienen sollen, wie die Arbeit mit dem Web durch unter-

¹<https://www.w3.org/WAI/people-use-web/user-stories/>

schiedlichste Behinderungen beeinflusst bzw. durch richtige Accessibility entsprechend erleichtert werden kann.

Das Gremium des W3C zur Förderung und Etablierung von Web Accessibility ist die *Web Accessibility Initiative* – kurz WAI – deren Aufgaben und Ziele im folgenden Abschnitt erläutert werden.

6.2 Die Web Accessibility Initiative

Die *Web Accessibility Initiative* [15] ist eine Arbeitsgruppe innerhalb des W3C, die sich ausschließlich mit dem barrierefreien Zugang zum Web beschäftigt.

6.2.1 Ziele

Die Ziele der WAI lassen sich wie folgt zusammenfassen:

- Entwicklung von Richtlinien, die als internationaler Standard für Web Accessibility angesehen werden,
- Veröffentlichung von Unterlagen, die zum Verständnis und zur Implementierung von Web Accessibility beitragen,
- Entwicklung von Accessibility Ressourcen durch internationale Zusammenarbeit.

Die dabei entstehenden Materialien durchlaufen dabei den üblichen W3C Veröffentlichungsmechanismus, wie er schon von Technologien wie etwa bei CSS bekannt ist. Die entstehenden Richtlinien beschäftigen sich jedoch nicht nur mit der Frage, wie Websites barrierefrei(er) gemacht werden können. Das Ziel lautet, den gesamten Prozess besser zu gestalten. Die entwickelten Richtlinien können dabei grob in drei verschiedene Kategorien eingeteilt werden:

- Richtlinien für Inhalte im Web (d. h. Websites),
- Richtlinien für Autor*innenwerkzeuge (d. h. Software, IDEs, CMS),
- Richtlinien für Clients (d. h. Webbrowser).

Innerhalb der WAI existieren einzelne Arbeitsgruppen, die sich mit der Erstellung der verschiedenen Richtlinien beschäftigen. Communitygruppen bieten darüber hinaus jedem*jeder die Möglichkeit, sich an Themen der Barrierefreiheit rund um die WAI zu beteiligen.

6.2.2 Arbeits- und Communitygruppen

Innerhalb der WAI existieren vier *Arbeitsgruppen*, die an verschiedenen Themen und Spezifikationen rum um Accessibility arbeiten. Im Folgenden wird ein kurzer Überblick über ihre Aufgaben gegeben. Über Communitygruppen können sich Interessierte an der WAI beteiligen, ohne direkt in einer Arbeitsgruppe mitzuarbeiten. Diese Gruppen sind jedoch nicht für die Erstellung von Standards zuständig, sondern dienen dem Austausch von Informationen und Ideen.

Accessibility Guidelines Working Group (AG WG)

Die wohl wichtigste – und meist mit dem Thema Accessibility direkt in Verbindung gebrachte – Arbeitsgruppe ist die *Accessibility Guidelines Working Group*². Sie veröffentlicht die Web Content Accessibility Guidelines (siehe Abschnitt 6.3), die die Referenz für Accessibility im Web darstellen.

Accessible Platform Architectures (APA) Working Group

Die *Accessible Platform Architectures Working Group*³, kurz APA, beschäftigt sich mit der Kontrolle von anderen W3C Spezifikationen hinsichtlich Accessibility auf einem technischen Level. Andere Arbeitsgruppen sollen dabei bei der Umsetzung von Accessibility Richtlinien unterstützt und mit technischem Know-How versorgt werden. Über einzelne, so genannte Task Forces werden spezielle Accessibility Brennpunkte gezielt bearbeitet. Zu den von dieser Arbeitsgruppe entwickelten Dokumenten gehören unter anderem die Synchronization Accessibility User Requirements [12] oder die XR Accessibility User Requirements [18]. Diese Dokumente beschreiben die Anforderungen, die Personen mit Einschränkungen in Bezug auf Synchronisierung in Audio und Video bzw. in Virtual und Augmented Reality haben.

Accessible Rich Internet Applications (ARIA) Working Group

Die *Accessible Rich Internet Applications (ARIA) Working Group*⁴, kurz ARIA WG, entwickelt Technologien, die bestehende Webinhalte für Personen mit Behinderungen im Bezug auf Accessibility verbessern sollen. Der Kern dabei sind die Richtlinien zu Accessible Rich Internet Applications (WAI-ARIA), welche in Abschnitt 6.4 vorgestellt werden. Einzelne Task Forces widmen sich innerhalb der Arbeitsgruppe verschiedenen Spezialthemen.

WAI Interest Group (WAI IG)

Die *WAI Interest Group*⁵, kurz WAI IG, produziert selbst keine Materialien zum Thema Accessibility sondern übernimmt die Revision und Diskussion von Richtlinien aus anderen Arbeitsgruppen. Darüber hinaus dient die Gruppe für den allgemeinen Informationsaustausch zum Thema Accessibility. Der Beitritt zu Mailinglisten oder das Folgen des Mastodon Feeds⁶ bietet einen guten Einblick über aktuelle Entwicklungen rund um das Thema Accessibility im W3C.

Community Gruppen

Neben den offiziellen Arbeitsgruppen gibt es auch *Community Gruppen*⁷, die sich mit speziellen Themen rund um Accessibility beschäftigen. Diese Gruppen sind offen für alle

²<https://www.w3.org/WAI/about/groups/agwg/>

³<https://www.w3.org/WAI/about/groups/apawg/>

⁴<https://www.w3.org/WAI/about/groups/ariawg/>

⁵<https://www.w3.org/WAI/about/groups/waiig/>

⁶<https://w3c.social/@wai>

⁷<https://www.w3.org/community/>

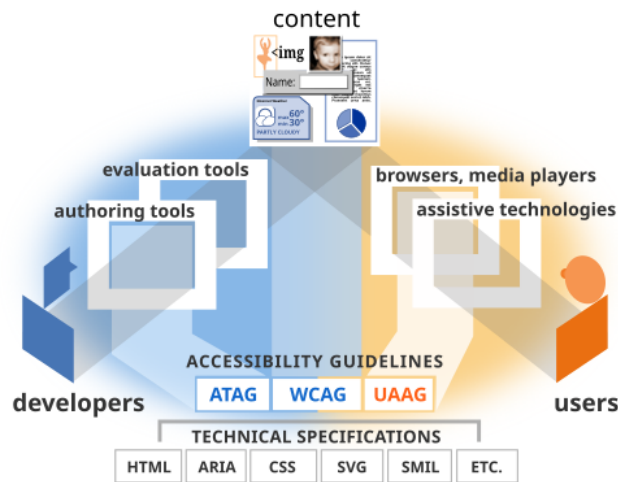


Abbildung 6.1: Essentielle Komponenten der Web-Accessibility und die Guidelines, die diese beschreiben. Quelle: [7].

Interessierten und bieten eine Plattform, um sich über spezielle Themen auszutauschen. Diese Gruppen sind für Webentwickler*innen und Interessierte gedacht, die sich für Accessibility interessieren und Ideen vorschlagen und diskutieren möchten. Die Gruppen sind dabei selbstorganisiert und können eigene Arbeitsgruppen bilden, um spezielle Themen zu bearbeiten.

6.2.3 Ergebnisse und Guidelines

Relevanter als die einzelnen Arbeitsgruppen sind die Ergebnisse, die von diesen hervor- gebracht werden. Analog zu den vom W3C spezifizierten Technologien, die als Recom- mendations den Stellenwert eines Standards erreichen, gibt es diesen Prozess auch für die einzelnen Richtlinien. Das W3C hat sich also zum Ziel gesetzt, konkrete Vorgaben im Bereich Accessibility herauszugeben, die es ermöglichen, die Anforderungen dieses Bereichs zu vereinheitlichen.

Während bei Accessibility jedoch zunächst meist nur wichtig erscheint, wie reguläre Websites für beeinträchtigte Personen leichter zugänglich gemacht werden können, ist der Bereich in Wirklichkeit jedoch viel umfangreicher. So ist es auch wichtig, Tools zur Entwicklung von Websites barrierefrei zu gestalten. Genauso sollen Webbrowser entspre- chend einfach zu bedienen sein und mit dem Aufkommen von dynamischen Webinhalten und User Interfaces ergeben sich erneut unterschiedliche Anforderungen.

Im folgenden wird zunächst ein Überblick über die wichtigsten Guidelines gegeben. Deren Zusammenhänge werden gut in Abbildung 6.1 dargestellt.

Web Content Accessibility Guidelines (WCAG)

Die *Web Content Accessibility Guidelines*⁸, kurz WCAG, sind Richtlinien zur barrierefreien Gestaltung von Webseiten. Sie sind mit Abstand die wichtigsten Regeln der WAI und werden daher in Abschnitt 6.3 gesondert im Detail erklärt.

Authoring Tools Accessibility Guidelines (ATAG)

Die *Authoring Tool Accessibility Guidelines*⁹ [11] sind Accessibility-Richtlinien, die sich mit der Gestaltung von so genannten Authoring Tools, also Software zum Erstellen von Webinhalten beschäftigen. Das W3C definiert den Begriff der Authoring Tools sehr weitläufig:

- Software, die zum Erzeugen von Webinhalten verwendet wird, z. B. WYSIWYG-Editoren wie Adobe Dreamweaver, oder auch JetBrains PhpStorm.
- Werkzeuge, die es ermöglichen, Dokumente in Web-Formaten abzuspeichern (z. B. Microsoft Word, das seine Dokumente auch als HTML-Datei speichern kann).
- Tools zum Erzeugen von Multimedia-Inhalten (Video, Audio etc.), die im Web zum Einsatz kommen.
- Content- und Site-Management-Systeme, die Webinhalte dynamisch aus Vorlagen (Templates) erzeugen können.
- Websites, die es Benutzer*innen ermöglichen, Inhalte hinzuzufügen (z. B. Blogs, Wikis, soziale Netze etc.).

Die ATAG-Richtlinien sind dabei in zwei *Teile* unterteilt. Teil A betrifft die Accessibility der Benutzer*innenoberfläche des Programms, Teil B betrifft die Generierung von barrierefreien Inhalten. Jeder Teil besteht aus *Prinzipien*, die die grobe Struktur vorgeben. Jedes der Prinzipien enthält wiederum die eigentlichen *Richtlinien*, also die Ziele, die von Authoring Tools erfüllt werden sollen, um als barrierefrei zu gelten. Für jede Guideline sind wiederum *Erfolgskriterien* definiert, die klar testbar sind und somit die schlussendliche Konformität zu ATAG 2.0 definieren. Hierfür gibt es drei Erfüllungsgrade: A, AA und AAA.

User Agent Accessibility Guidelines (UAAG)

Die *User Agent Accessibility Guidelines*¹⁰ [13], kurz UAAG, sind Richtlinien, die sich mit der barrierefreien Umsetzung von so genannten User Agents, also Software zur Darstellung von Webinhalten beschäftigen. Diese Richtlinien sind in erster Linie für Entwickler*innen von Webbrowsern, Media Playern oder unterstützenden Technologien wie Screen Readern wichtig.

Die Richtlinien sind dabei in fünf *Prinzipien* unterteilt, welche die grobe Struktur vorgeben. Jedes der Prinzipien enthält wiederum die eigentlichen *Richtlinien*, also die Ziele, die von User Agents erfüllt werden sollen, um als barrierefrei zu gelten. Für jede Guideline sind wiederum *Erfolgskriterien* definiert, die klar testbar sind und somit die

⁸<https://www.w3.org/WAI/standards-guidelines/wcag/>

⁹<https://www.w3.org/WAI/standards-guidelines/atag/>

¹⁰<https://www.w3.org/WAI/standards-guidelines/uaag/>

schlussendliche Konformität zu UAAG 2.0 definieren. Hierfür gibt es drei Erfüllungsgrade: A, AA und AAA.

Accessible Rich Internet Applications (WAI-ARIA)

Moderne Websites greifen vermehrt auf dynamische Elemente zurück, die es ermöglichen, neue Inhalte nachzuladen, ohne die Seite neu zu laden, Drag & Drop im Browser zu verwenden, oder grafische Effekte wie Ein- und Ausklappen von Elementen zu realisieren. Man spricht hierbei auch von so genannten *Rich Internet Applications*, kurz RIA. Für sie hat das W3C eine eigene Spezifikation veröffentlicht: *WAI-ARIA*¹¹. Diese wird in Abschnitt 6.4 ausführlich beschrieben.

6.3 Web Content Accessibility Guidelines (WCAG)

Die *Web Content Accessibility Guidelines* [16], kurz WCAG und zu deutsch Richtlinien für barrierefreie Webinhalte, sind ein Standard des W3C zur barrierefreien Gestaltung von Web Inhalten.

6.3.1 Allgemeines

Die WCAG sind mit Abstand die bekanntesten Richtlinien im Bereich der Accessibility. Oftmals werden sie auch stellvertretend für den Bereich Accessibility im Web angesehen, was allerdings nicht stimmt. Wie in den vorigen Abschnitten erwähnt, existieren auch Guidelines für andere Bereiche wie User Agents oder Authoring Tools, die ebenso wichtig, aber für „die breite Masse“ im Regelfall weniger relevant sind.

Die Notwendigkeit, Richtlinien zur barrierefreien Gestaltung von Webinhalten zu erstellen, kam mit der zunehmenden Verbreitung des World Wide Web in den 1990er Jahren auf. Während die Anfänge des Web nur textbasiert waren, kamen zunehmend Multimediainhalte auf, die für beeinträchtigte Personen nicht mehr zugänglich waren. Zusätzlich wurde diese Problematik einerseits durch die fehlende Standardkonformität der Browser und andererseits durch mangelhaftes Problembewusstsein bei Webautor*innen verschärft. Letzteres äußert sich in der Formulierung von Inhalten, die primär aufgrund visueller Gegebenheiten erfolgen (z. B. ein Link mit dem Text „hier klicken“ – dieser Link bietet keinerlei Informationen im Text, was sich dahinter verbirgt).

HTML bietet zwar in seiner aktuellen Version, d. h. dem Living Standard, sowie in seinen Vorgängerversionen HTML 4.01 bzw. auch XHTML 1.0 genügend Möglichkeiten, Barrierefreiheit zu realisieren, jedoch ist dies ohne konkrete Vorgaben und Anleitungen ein schwieriges Unterfangen. Webautor*innen müssen mit der Thematik vertraut gemacht werden, nur dann ist eine weit verbreitete Umsetzung möglich. Zu diesem Zweck wurden die WCAG entworfen, die aktuell in Version 2.2 vom Dezember 2024 als Recommendation vorliegen und seit September 2025 auch ein offizieller ISO/IEC Standard sind [3]. Die Arbeit an den WCAG 3.0 wurde Ende 2021 gestartet und befindet sich derzeit noch in einem sehr frühen Stadium [14].

¹¹<https://www.w3.org/WAI/standards-guidelines/aria/>

6.3.2 WCAG 2.2

Die *Web Content Accessibility Guidelines (WCAG) 2.2* [16] sind eine Weiterentwicklung der Version 2.1, wobei 9 neue Erfolgskriterien hinzukamen und ein obsoletes entfernt wurde. Die Änderungen ergeben sich hierbei in der Regel durch die Weiterentwicklung der Webtechnologien, die neue Erfordernisse mit sich bringen. Die WCAG 2.2 Kriterien sind sowohl maschinell als auch durch menschliche Evaluierung testbar – ersteres ist nicht immer mit jedem Tool sofort möglich, da viele oft länger benötigen, bis der aktuelle Standard inkludiert ist.

Wie auch schon zuvor bei den ATAG 2.0 und UAAG 2.0 zu sehen war, existieren auch für WCAG 2.2 vier große *Prinzipien*, welche wiederum mit *Richtlinien* unterlegt sind. Jede dieser Guidelines hat testbare Erfolgskriterien in drei verschiedenen Ebenen (A, AA, AAA), die bestimmen, wie sehr eine Website zu WCAG 2.2 konform ist:

- **Stufe A:** Level A Konformität stellt das Minimum an Konformität dar. Alle Erfolgskriterien auf Ebene A sind erfüllt.
- **Stufe AA:** Level AA Konformität bedeutet, dass alle Level A und Level AA Erfolgskriterien erfüllt sind.
- **Stufe AAA:** Level AAA Konformität wird durch die Erfüllung aller A, AA und AAA Erfolgskriterien erreicht.

WCAG 2.2 Guidelines

Die vier zugrunde liegenden Prinzipien sind im Englischen unter dem Akronym **POUR** zusammengefasst: **P**erceivable (wahrnehmbar), **O**perable (bedienbar), **U**nderstandable (verständlich), **R**obust. Im Folgenden sind die vier Prinzipien mit den dazugehörigen Guidelines und einigen wichtigen Erfolgskriterien aufgeführt.

1. **Prinzip 1: Perceivable/Wahrnehmbar** – Informationen und Bestandteile der Benutzer*innenschnittstelle müssen den Benutzer*innen so präsentiert werden, dass diese sie wahrnehmen können.
 - **Richtlinie 1.1: Textalternativen:** Stelle Textalternativen für alle Nicht-Text-Inhalte zur Verfügung, so dass diese in andere, von Benutzer*innen benötigte Formen geändert werden können, wie zum Beispiel Großschrift, Braille, Symbole oder einfachere Sprache.
 - Alle Nicht-Text-Inhalte, die den Benutzer*innen präsentiert werden, haben eine Textalternative, die einem gleichen Zweck dient (STUFE A).
 - **Richtlinie 1.2: Zeitbasierte Medien:** Stelle Alternativen für zeitbasierte Medien zur Verfügung.
 - Es werden Alternativen für reine Audio- und Videoinhalte bereit gestellt (STUFE A).
 - Untertitel werden für alle aufgezeichneten Audioinhalte in synchronisierten Medien bereitgestellt (STUFE A).
 - ...
 - **Richtlinie 1.3: Anpassbar:** Erstelle Inhalte, die auf verschiedene Arten

dargestellt werden können (z. B. einfacheres Layout), ohne dass Informationen oder Struktur verloren gehen.

- Informationen, Struktur und Beziehungen, die über die Darstellung vermittelt werden, können durch Software bestimmt werden oder stehen in Textform zur Verfügung (STUFE A).
- Wenn die Reihenfolge, in der Inhalte präsentiert werden, sich auf deren Bedeutung auswirkt, kann die korrekte Leseabfolge durch Software bestimmt werden (STUFE A).
- Anweisungen, die für das Verständnis und die Bedienung von Inhalt bereitgestellt werden, stützen sich nicht nur auf sensorische Eigenschaften von Komponenten wie Form, Größe, visuelle Position, Ausrichtung oder Ton (STUFE A).
- Der Inhalt der Seite wird nicht auf eine einzelne Orientierung der Darstellung (Hoch- oder Querformat) reduziert, außer dies ist absolut notwendig (STUFE AA).
- ...
- **Richtlinie 1.4: Unterscheidbar:** Mache es Benutzer*innen leichter, Inhalt zu sehen und zu hören einschließlich der Trennung von Vorder- und Hintergrund.
 - Farbe wird nicht als einziges visuelles Mittel benutzt, um Informationen zu vermitteln, eine Handlung zu kennzeichnen, eine Reaktion zu veranlassen oder ein visuelles Element zu unterscheiden (STUFE A).
 - Die visuelle Darstellung von Text und Bildern von Text hat ein Kontrastverhältnis von mindestens 4,5:1 (STUFE AA).
 - ...

2. Prinzip 2: Operable/Bedienbar – Bestandteile der Navigation und Benutzer*innenschnittstelle müssen bedienbar sein.

- **Richtlinie 2.1: Per Tastatur zugänglich:** Sorge dafür, dass alle Funktionalitäten per Tastatur zugänglich sind.
 - Alle Funktionalitäten des Inhalts sind durch eine Tastaturschnittstelle bedienbar, ohne dass eine bestimmte Zeiteinteilung für einzelne Tastenanschläge erforderlich ist (STUFE A).
 - Sind Keyboard-Shortcuts vorhanden, die nur aus Buchstaben, Zahlen oder Punktionszeichen bestehen, müssen diese umbelegt, deaktiviert oder nur aktiv sein, wenn diese Komponente den Fokus besitzt (STUFE A).
 - ...
- **Richtlinie 2.2: Ausreichend Zeit:** Gib den Benutzer*innen ausreichend Zeit, Inhalte zu lesen und zu benutzen.
 - Benutzer*innen können die zeitliche Begrenzung abschalten, anpassen oder ausweiten, bevor sie darauf treffen (STUFE A).
 - Benutzer*innen können bewegende, blinkende oder scrollende Informationen anhalten oder pausieren (STUFE A).

- Benutzer*innen werden über die maximal mögliche Dauer von Inaktivität gewarnt, falls es dadurch zu Datenverlust kommen kann (STUFE AAA).
 - ...
 - **Richtlinie 2.3: Anfälle und körperliche Reaktionen:** Gestalte Inhalte nicht auf Arten, die bekanntermaßen Anfälle oder körperliche Reaktionen auslösen können.
 - Webseiten enthalten nichts, was öfter als dreimal in einem beliebigen, eine Sekunde dauernden Zeitraum blitzt, oder der Blitz ist unterhalb der allgemeinen Grenzwerte zu Blitzen und roten Blitzen (STUFE A).
 - Animationen, die durch Interaktion mit der Seite hervorgerufen werden, können deaktiviert werden, außer diese Interaktion ist für die Funktionalität essentiell (STUFE AAA).
 - ...
 - **Richtlinie 2.4: Navigierbar:** Stelle Mittel zur Verfügung, um Benutzer*innen dabei zu unterstützen zu navigieren, Inhalte zu finden und zu bestimmen, wo sie sich befinden.
 - Es gibt einen Mechanismus, um Inhaltsblöcke zu umgehen, die auf verschiedenen Webseiten wiederholt werden (STUFE A).
 - Webseiten haben einen Titel, der Thema oder Zweck beschreibt (STUFE A).
 - Überschriften und Labels beschreiben ein Thema oder einen bestimmten Zweck (STUFE AA).
 - ...
 - **Richtlinie 2.5: Eingabemodalitäten:** Mache es für Benutzer*innen einfacher, Funktionalitäten mit anderen Eingabemodalitäten außer dem Keyboard zu verwenden.
 - Alle Funktionalitäten, die Multi-Point oder pfadbasierte Gesten verwenden, sind auch durch einen Single-Pointer (Tap, Click, Double-Tap oder Long-Press) erreichbar (STUFE A).
 - Alle Komponenten der Benutzer*innenschnittstelle, die mit einem Label beschriftet sind, enthält der (programmatische) Name den Text, der visuell dargestellt wird. (STUFE A).
 - ...
3. **Prinzip 3: Understandable/Verständlich** – Informationen und Bedienung der Benutzer*innenschnittstelle müssen verständlich sein.
- **Richtlinie 3.1: Lesbar:** Mache den Textinhalt lesbar und verständlich.
 - Die voreingestellte menschliche Sprache jeder Webseite kann durch Software bestimmt werden (STUFE A).
 - Es gibt einen Mechanismus, um spezielle Definitionen von Wörtern oder Wendungen zu erkennen, die auf ungewöhnliche oder eingeschränkte Weise benutzt werden, Idiome und Jargon eingeschlossen (STUFE AAA).

- Es gibt einen Mechanismus, um die ausgeschriebene Form oder Bedeutung von Abkürzungen zu erkennen (STUFE AAA).
 - ...
 - **Richtlinie 3.2: Vorhersehbar:** Sorge dafür, dass Webseiten vorhersehbar aussehen und funktionieren.
 - Wenn irgendein Bestandteil den Fokus erhält, dann löst dies nicht eine Änderung des Kontextes aus (STUFE A).
 - Navigationsmechanismen, die auf mehreren Webseiten innerhalb eines Satzes von Webseiten wiederholt werden, treten jedes Mal, wenn sie wiederholt werden, in der gleichen relativen Reihenfolge auf (STUFE AA).
 - Wenn eine Webseite einen Hilfemechanismus enthält und dieser auf mehreren Webseiten innerhalb einer Reihe von Webseiten wiederholt wird, erscheint er in der gleichen Reihenfolge in Bezug auf andere Seiteninhalte [...] (STUFE A).
 - ...
 - **Richtlinie 3.3: Hilfestellung bei der Eingabe:** Hilf den Benutzer*innen dabei, Fehler zu vermeiden und zu korrigieren.
 - Wenn ein Eingabefehler automatisch erkannt wird, dann wird das fehlerhafte Element identifiziert und der Fehler wird den Benutzer*innen in Textform beschrieben (STUFE A).
 - Wenn der Inhalt eine Eingabe durch die Benutzer*innen verlangt werden Beschriftungen (Labels) oder Anweisungen bereitgestellt (STUFE A).
 - Es gibt eine kontextsensitive Hilfe (STUFE AAA).
 - Informationen, die Nutzer*innen zuvor eingegeben haben oder die ihnen zur Verfügung gestellt wurden und die im selben Vorgang erneut eingegeben werden müssen, werden entweder automatisch ausgefüllt, oder für die Benutzer*innen zur Auswahl verfügbar [...] (STUFE A).
 - ...
4. **Prinzip 4: Robust** – Inhalte müssen so robust sein, dass sie von einer Vielzahl von User Agents, einschließlich assistiver Technologien, interpretiert werden können.
- **Richtlinie 4.1: Kompatibel:** Maximiere die Kompatibilität mit aktuellen und zukünftigen User Agents, einschließlich assistiver Technologien.
 - Für alle Komponenten der Benutzeroberfläche (einschließlich, aber nicht beschränkt auf: Formularelemente, Links und von Skripten generierte Komponenten) können Name und Rolle programmatisch festgelegt werden; Zustände, Eigenschaften und Werte, die von Benutzer*innen eingestellt werden können, können programmatisch eingestellt werden; und die Benachrichtigung über Änderungen an diesen Elementen ist für User Agents, einschließlich assistiver Technologien, verfügbar (STUFE A).
 - In Inhalten, die mit Auszeichnungssprachen implementiert wurden, können Statusmeldungen programmatisch durch Rollen oder Eigenschaften

bestimmt werden, so dass sie dem Nutzer von assistiven Technologien präsentiert werden können, ohne dass sie den Fokus erhalten (STUFE AA).

– ...

Dokumente zur Umsetzung

Zusätzlich zu den WCAG 2.2 Richtlinien existieren Dokumente die bei der Umsetzung unterstützen:

- *How to Meet WCAG (Quick Reference)*¹²,
- *Understanding WCAG 2.2*¹³,
- *Techniques for WCAG 2.2*¹⁴.

6.3.3 Evaluierung mit den WCAG 2.2

Die eigene Webseite auf Accessibility-Probleme zu überprüfen ist (leider) kein komplett automatisierbarer Prozess, sondern erfordert primär das Verständnis und das Hintergrundwissen von Entwickler*innen. Das W3C stellt jedoch Materialien und Tools zur Verfügung, die den Prozess vereinfachen und vor allem anleiten können.

Grobe Evaluierung mit Easy Checks

Die Seite *Easy Checks*¹⁵ stellt die häufigsten und am schnellsten behebbaren Probleme mit Accessibility übersichtlich dar. Die Rationale hinter dieser Liste ist, dass sie schnell abzuarbeiten ist somit grundsätzliche Probleme einfach eliminieren kann. Trotzdem kann die eigene Seite nach dem Abarbeiten der Easy Checks-Liste noch Accessibility-Probleme aufweisen, weshalb eine genauere Evaluierung sinnvoll ist.

Easy Checks besteht aus dreizehn Punkten, unterteilt in drei Kategorien, die auf der zugehörigen Seite mit Beispielen und technischen Tipps erläutert werden, sowie Bookmarklets zum Überprüfen beinhalten. Die Liste ist in Tabelle 6.1 dargestellt.

Detaillierte Evaluierung mit WCAG-EM

Die *Website Accessibility Conformance Evaluation Methodology* (WCAG-EM) [17] ist ein vom W3C strukturierter Prozess, um die Accessibility von Webseiten zu evaluieren. Er besteht aus fünf Schritten, die in Abbildung 6.2 visualisiert sind und im Folgenden kurz erklärt werden.

1. **Den Bereich der Evaluierung festlegen:** Welche *Seite* wird nach welchem *WCAG Erfüllungsgrad* in welchem *Endgerät* und mit welchen optionalen *Zusatzkriterien* evaluiert?

¹²<https://www.w3.org/WAI/WCAG22/quickref/>

¹³<https://www.w3.org/WAI/WCAG22/Understanding/>

¹⁴<https://www.w3.org/WAI/WCAG22/Techniques/>

¹⁵<https://www.w3.org/WAI/test-evaluate/easy-checks/>

Tabelle 6.1: Die Easy Checks-Liste des W3C mit ihren 13 Punkten.

Allgemeine Checks	
Alternativtext für Bilder	Jedes Bild benötigt eine aussagekräftige Textalternative.
Seitentitel	Die Seite soll einen sinnvollen und beschreibenden Titel aufweisen.
Überschriften	Überschriften sollen korrekt verwendet werden und eine sinnvolle Outline generieren.
Kontraste	Der Kontrast zwischen Vorder- und Hintergrund soll ausreichend sein. Die Darstellung muss auch in Graustufen noch funktionieren.
Skip Link	Die Seite hat einen Skip Link als erstes interaktives Element, um direkt zum Hauptinhalt zu springen.
Sichtbarer Keyboard-Fokus	Jedes interaktive Element auf der Seite soll ein offensichtliches, visuelles Styling aufweisen, wenn es den Fokus erhält.
Sprache der Seite	Die Sprache der Seite soll korrekt (mit dem <code>lang</code> -Attribut beim <code><html></code> -Element) deklariert sein.
Zoom	Die Seite soll auch bei 200 % Zoomfaktor noch korrekt dargestellt werden.
Audiovisuelle Checks	
Untertitel	Videos weisen Untertitel auf und diese sind von guter Qualität.
Transkripte	Medien (Videos und Audio) weisen ein Transkript auf, das die Inhalte verschriftlich darstellt.
Audiobeschreibung	Videos weisen eine Audiobeschreibung auf, die die visuellen Inhalte beschreibt.
Formular-Checks	
Labels	Formularelemente sind mit Labels versehen.
Pflichtfelder	Pflichtfelder sind klar gekennzeichnet, etwa mit einem Sternchen.

- (a) **Seite:** Welche Webseite wird überhaupt evaluiert und welche Art von Unterseiten (d. h. Webpages) davon werden inkludiert und welche nicht? Mobile Seiten, verschiedene Sprachversionen etc.
- (b) **Erfüllungsgrad:** Wird nach Level A, AA oder AAA evaluiert?
- (c) **Endgerät:** In welchem Browser wird getestet? Kommt ein Screenreader zum Einsatz, wenn ja, welcher?

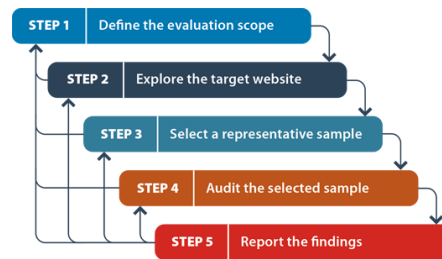


Abbildung 6.2: Der standardisierte Accessibility-Evaluationsprozess, definiert durch das W3C. Quelle: [17].

- (d) **Zusatzkriterien:** Werden nur Fehler aufgezeigt oder Lösungen angeboten? Gibt es spezielle Anwendungsfälle, die berücksichtigt werden müssen etc.
- 2. **Die Webseite untersuchen:** Die Struktur der Seite soll untersucht und verstanden werden. Welche *Unterseiten* gibt es? Welche *Funktionalitäten* bietet die Seite? Welche *Typen* von Seiten gibt es und welche *Technologien* werden verwendet?
 - (a) **Wichtige Unterseiten:** Welche wichtigen Unterseiten hat die aktuelle Webseite? Diese sind in der Regel über die Navigation erreichbar.
 - (b) **Funktionalitäten:** Was kann auf diesen Seiten getan werden? Zum Beispiel ein Produkt in einem Shop kaufen, Formulare ausfüllen und absenden oder sich registrieren.
 - (c) **Typen:** Welche Arten von Seiten gibt es? Zum Beispiel Seiten mit unterschiedlicher Struktur, Seiten mit verschiedenen Technologien, Seiten mit unterschiedlichen Interface Elementen etc.
 - (d) **Technologien:** Welche Technologien wurden verwendet? Etwa HTML, CSS, JavaScript, PDF, SVG etc.
- 3. **Ein repräsentatives Sample erstellen:** Nicht alle Seiten können oft evaluiert werden. Daher muss ein *Sample erstellt* werden, das Pages umfasst die aus den wichtigen Unterseiten mit den entsprechenden Funktionalitäten und Typen sowie Technologien besteht. Zusätzlich wird ein *zufälliges Sample* erstellt. Gibt es *komplette Prozesse*, müssen diese extra zusammengestellt werden.
 - (a) **Zusammengestelltes Sample:** Dieses muss alle in Schritt 2 definierten Teile (wichtige Seiten, Funktionalitäten, Typen und Technologien) enthalten, um repräsentativ zu sein.
 - (b) **Zufälliges Sample:** Um zu verifizieren, dass das zusammengestellte Sample repräsentativ ist, werden zusätzlich noch Seiten zufällig ausgewählt. Die Ergebnisse beider Samples sollten korrelieren.
 - (c) **Komplette Prozesse:** Gibt es komplette Prozesse, wie etwa einen Bestellvorgang in einem Onlineshop muss das Sample alle Teilseiten davon beinhalten.
- 4. **Das Sample prüfen:** Alle Seiten in den Samples werden nun anhand der WCAG-Kriterien *geprüft*. Beim zufälligen Sample dürfen keine neuen Typen, Technologien

oder Funktionalitäten erkannt werden, da sonst das zusammengestellte Sample nicht vollständig wäre. Komplette Prozesse müssen vollständig evaluiert werden.

- (a) **Prüfen:** Mit Hilfe der WCAG-Richtlinien bzw. der Quick Reference *How to Meet WCAG 2.2*¹⁶ werden die einzelnen Erfolgskriterien, die für den Erfüllungsgrad (A, AA oder AAA) relevant sind, überprüft.
- 5. **Die Resultate dokumentieren:** Die einzelnen Schritte müssen dokumentiert und das Ergebnis aufbereitet werden. Ein solcher standardisierter Bericht kann mit dem *WCAG-EM Report Tool*¹⁷ erstellt werden und enthält alle wichtigen Daten aus den einzelnen Schritten.

6.4 Accessible Rich Internet Applications (WAI-ARIA)

Die *Accessible Rich Internet Applications* [9] sind Richtlinien, die ein Framework zur Verbesserung der Accessibility von Webinhalten bereitstellen.

6.4.1 Allgemeines

Webseiten werden immer moderner und benutzer*innenfreundlicher. Statische Seiten werden mehr und mehr durch dynamische Ansätze ersetzt, neue und interaktive Elemente – oft Widgets genannt – ermöglichen Drag & Drop, zeigen Tabs auf einer Webseite an oder aktualisieren Teile der Webseite im Hintergrund, ohne diese neu laden zu müssen.

Für derartige Effekte ist fast ausnahmslos immer eine Skriptsprache wie JavaScript nötig, mit der eine fertig geladene HTML-Seite im Nachhinein noch dynamisch verändert werden kann. So können etwa neue Elemente erzeugt oder andere verändert und entfernt werden. Mit Hilfe verschiedener, vordefinierter JavaScript Komponenten (aus Frameworks wie etwa Bootstrap¹⁸) können etwa auch spezielle Elemente in eine Webpage integriert werden, die mit „HTML-Bordmitteln“ nicht möglich wären.

Diese Flexibilität geht jedoch sehr stark auf Kosten der Accessibility, da es für unterstützende Technologien normal nicht möglich ist, diese dynamischen Änderungen mitzuverfolgen und entsprechend darauf zu reagieren. Somit ist es etwa für einen Screenreader nicht möglich, dynamisch generierte Inhalte vorzulesen, da sie dieser schlichtweg nicht erkennt. Selbst wenn die Inhalte erkannt werden, so fehlen oft jedoch semantische Informationen, die es dem Screenreader ermöglichen zu erkennen, ob es sich bei dem generierten Inhalt etwa um ein Menü, einen Slider oder ein völlig anderes Element handelt. Nur wenn dies bekannt ist, kann der Screenreader das Element entsprechend behandeln. Die *WAI-ARIA*¹⁹ schaffen hierfür Abhilfe.

6.4.2 WAI-ARIA 1.2

Die Spezifikation zu *Accessible Rich Internet Applications 1.2* [9] ist seit Juni 2023 eine W3C Recommendation (WAI-ARIA 1.3 [10] ist noch im Status eines Working Drafts

¹⁶<https://www.w3.org/WAI/WCAG22/quickref/>

¹⁷<https://www.w3.org/WAI/eval/report-tool/>

¹⁸<https://getbootstrap.com/>

¹⁹<https://www.w3.org/WAI/standards-guidelines/aria/>

und wurde erst begonnen). Sie beschreibt ein Accessibility Framework, das Webentwickler*innen eine Reihe von *Rollen*, *Zuständen* und *Eigenschaften* bietet, um dynamischen Inhalten Bedeutungen zu geben. Diese Eigenschaften werden als HTML-Attribute in bestehendes Markup integriert und erlauben so den unterstützenden Technologien, darauf zu reagieren.

Rollen

Rollen werden mit dem Attribut **role** hinzugefügt und werden in fünf Kategorien eingeteilt:

Widget-Rollen: Diese Rollen kennzeichnen Elemente als standalone Widgets oder als Teile von größeren, zusammengesetzten Widgets. Folgende Rollen sind verfügbar:

- **button** (Schaltfläche),
- **checkbox** (Checkbox),
- **gridcell** (Rasterzelle),
- **link** (Link),
- **menuitem** (Option in einem Menü),
- **menuitemcheckbox** (Checkbox in einem Menü),
- **menuitemradio** (ein Radiobutton in einem Menü),
- **option** (Option in einer Auswahlliste),
- **progressbar** (Fortschrittsanzeige),
- **radio** (Radiobutton),
- **scrollbar** (Scrollleiste),
- **searchbox** (Suchfeld),
- **separator** (Trennlinie),
- **slider** (Slider),
- **spinbutton** (Schaltfläche zum Wählen diskreter Nummernwerte),
- **switch** (binäre Ein/Aus-Checkbox),
- **tab** (Tab/Reiter),
- **tabpanel** (Behälter für Tabs),
- **textbox** (Textbox) und
- **treeitem** (Option in einem Baum).

Diese Elemente sind in oft in anderen, so genannten Container-Elementen enthalten. Für diese gibt es die folgenden Rollen:

- **combobox** (Kombinationselement aus Textbox und z. B. Listbox),
- **grid** (Raster),
- **listbox** (Auswahl mehrere Einträge aus einer Liste),
- **menu** (Auswahlmenü),
- **menubar** (Menüleiste, enthält meist das Menü),
- **radiogroup** (Gruppe von Radiobuttons),
- **tablist** (Container für mehrere Tab-Panels),

- **tree** (Baumstruktur, die aus- und einklappbar ist) und
- **treegrid** (Raster, der aus- und einklappbar ist).

Dokumenten-Rollen: Diese Rollen beschreiben Strukturen, die Inhalte auf einer Seite organisieren. Dokumentstrukturen sind normalerweise nicht interaktiv:

- **application** (generelle Applikation, die nicht durch einen Widget abgedeckt ist),
- **article** (eigenständiger Artikel),
- **blockquote** (Zitat aus einer anderen Quelle),
- **caption** (Beschriftung für eine Abbildung, Tabelle, Grid oder Treegrid),
- **cell** (Tabellenzelle),
- **columnheader** (Header für eine Tabellenspalte),
- **definition** (Definition eines Begriffs),
- **deletion** (als entfernt markierte, oder zur Entfernung vorgeschlagene Inhalte),
- **directory** (Verzeichnis z. B. von Personen),
- **document** (Dokument, das (vor-)gelesen werden soll),
- **emphasis** (betonter Inhalt),
- **feed** (Liste an Beiträgen),
- **figure** (Abbildung),
- **generic** (namenloses Containerelement ohne semantische Bedeutung),
- **group** (Set an UI-Elementen, die nicht vom Screenreader vorgelesen werden sollen),
- **heading** (Überschrift),
- **img** (Bild),
- **insertion** (als hinzugefügt markierte Inhalte),
- **list** (Liste),
- **listitem** (Element einer Liste),
- **math** (mathematischer Ausdruck),
- **meter** (Maß innerhalb eines skalen Bereichs),
- **none** (Element wird von unterstützenden Technologien ignoriert),
- **note** (Anmerkung),
- **paragraph** (Absatz mit Inhalten),
- **presentation** (Synonym für **none**),
- **row** (Tabellenreihe),
- **rowgroup** (mehrere Reihen einer Tabelle),
- **rowheader** (Header für eine Tabellenreihe),
- **separator** (Trennlinie),
- **strong** (wichtige, ernste oder dringende Inhalte),
- **subscript** (tiefgestellte Zeichen),
- **superscript** (hochgestellte Zeichen),
- **table** (Tabelle),
- **term** (Ausdruck mit Definition),

- **time** (Datum und/oder Uhrzeit),
- **toolbar** (Toolbar) und
- **tooltip** (Tooltip-Popup).

Landmark-Rollen: Diese Rollen beschreiben wichtige Bereiche auf der Seite, zu denen User*innen leicht Zugriff bekommen sollten. Folgende gibt es dabei:

- **banner** (Seitenweiter Bereich etwa mit einem Logo),
- **complementary** (Zusatzinformationen),
- **contentinfo** (Informationen zum Dokument wie Copyright),
- **form** (Formular),
- **main** (Hauptbereich),
- **navigation** (Navigation),
- **region** (genereller Abschnitt) und
- **search** (Suche).

Live Region-Rollen: Diese Rollen kennzeichnen Bereiche auf der Seite, die sich dynamisch ändern können:

- **alert** (wichtige Informationen und Meldungen),
- **log** (Live-Log bei dem neue Informationen erscheinen und alte verschwinden),
- **marquee** (schnell ändernde, unwichtige Informationen),
- **status** (weniger wichtige Statusinformationen) und
- **timer** (Timer oder Countdown).

Window-Rollen: Diese Rollen definieren Fenster innerhalb des Browsers:

- **alertdialog** (Alert-Dialogfenster) und
- **dialog** (Dialogfenster).

Anwendung von Rollen

Rollen kommen dort zum Einsatz, wo semantische HTML-Elemente nicht verfügbar sind oder nicht entsprechend eingesetzt werden können. So gibt es keine HTML-Elemente um Tabs in einer Seite anzuzeigen. Das Markup hierfür könnte mit Hilfe einer geordneten Liste erstellt werden. Diese hat allerdings keinerlei semantische Informationen über Tabs, daher werden die entsprechenden WAI-ARIA Rollen bei den Elementen hinzugefügt. **tablist** beim `<ol>`-Element kennzeichnet die gesamte Tab-Liste, jedes `<li>`-Element wird mittel **tab** zum Reiter und die `<div>`-Elemente mit **tabpanel** werden als Inhaltsbereiche des Tabs ausgewiesen. Abbildung 6.3 zeigt eine mögliche Darstellung der Tabs.

```

1 <ol role="tablist">
2   <li id="ch1Tab" role="tab">
3     <a href="#ch1Panel">Chapter 1</a>
4   </li>
5   <li id="ch2Tab" role="tab">
6     <a href="#ch2Panel">Chapter 2</a>

```

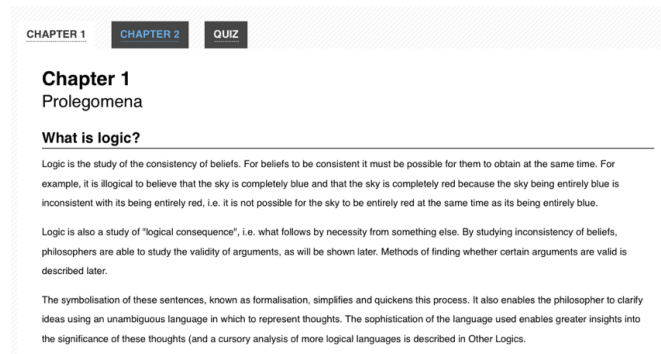


Abbildung 6.3: Visuell sind die Tabs sofort erkennbar. Im Markup müssen WAI-ARIA-Rollen nachhelfen. Quelle: [6].

```

7   </li>
8   <li id="quizTab" role="tab">
9     <a href="#quizPanel">Quiz</a>
10  </li>
11 </ol>
12
13 <div>
14   <div id="ch1Panel" role="tabpanel" aria-labelledby="ch1Tab">Chapter 1 content goes
15     here</div>
16   <div id="ch2Panel" role="tabpanel" aria-labelledby="ch2Tab">Chapter 2 content goes
17     here</div>
18   <div id="quizPanel" role="tabpanel" aria-labelledby="quizTab">Quiz content goes
19     here</div>
20 </div>

```

Zustände und Eigenschaften

Zustände und Eigenschaften sind Attribute (beginnend mit `aria-`), die zu HTML-Elementen hinzugefügt werden und ergänzen Rollen durch Zusatzinformationen. Sie sind im Konzept sehr ähnlich, Zustände sind jedoch Attribute deren Wert sich ändern kann (z. B. `aria-checked="true"` zu `false`, um den Zustand einer Checkbox anzugeben), während Eigenschaften in der Regel unverändert bleiben (z. B. `aria-labelledby`, um zu beschreiben, durch welches Element ein anderes beschriftet wird – siehe dazu Code zu den Tabs im Abschnitt vorher). Generell hat diese Unterscheidung jedoch keine Konsequenzen, weshalb hier auch einfach von ARIA-Attributen gesprochen wird.

Attribute werden in vier Kategorien eingeteilt:

Widget-Attribute: Diese Attribute finden sich in der Regel bei Widgets, d. h. Elementen mit den entsprechenden Rollen. Die Attributwerte werden hier nicht aufgelistet und können aus der WAI-ARIA Spezifikation [9] entnommen werden:

- `aria-autocomplete` (Element kann Autocompletion vornehmen),
- `aria-checked` (Element ist angehakt oder nicht),
- `aria-disabled` (Element ist deaktiviert oder nicht),

- **aria-errormessage** (Element stellt eine Fehlermeldung dar),
- **aria-expanded** (Widget ist zu- oder aufgeklappt),
- **aria-haspopup** (Element zeigt ein Popup an),
- **aria-hidden** (Element ist vor der unterstützten Technologie versteckt),
- **aria-invalid** (eingegebener Wert ist ungültig),
- **aria-label** (Element ist ein Label für ein anderes),
- **aria-level** (definierte die Hierarchietiefe eines Elements),
- **aria-modal** (Element ist ausschließlich modal, d. h. immer im Vordergrund),
- **aria-multiline** (mehrzeilige Eingaben sind möglich oder nicht),
- **aria-multiselectable** (mehrere Elemente können gleichzeitig gewählt werden),
- **aria-orientation** (Ausrichtung eines Elements),
- **aria-placeholder** (Platzhaltertext),
- **aria-pressed** (Button ist gedrückt),
- **aria-readonly** (Element ist nicht editierbar),
- **aria-required** (Element muss ausgefüllt werden),
- **aria-selected** (Element ist ausgewählt),
- **aria-sort** (Sortierung eines Widgets),
- **aria-valuemax** (der maximal erlaubte Eingabewert in einem Feld),
- **aria-valuemin** (der minimal erlaubte Eingabewert),
- **aria-valuenow** (der aktuell eingegebene Wert) und
- **aria-valuetext** (textuelle Beschreibung des eingegebenen Werts).

Live-Region-Attribute: Diese Attribute werden im Umgang mit Live-Regions, d. h. Bereiche deren Inhalte sich aufgrund von externen Ereignissen ändern können, verwendet:

- **aria-atomic** (assistive Technologien lesen alles oder nur die Änderungen neu vor),
- **aria-busy** (Element wird gerade verändert),
- **aria-live** (Element wird mit Inhalten aktualisiert werden) und
- **aria-relevant** (assistive Technologien lesen nur relevante Änderungen vor).

Drag-und-Drop-Attribute: Diese Attribute sind speziell für Drag & Drop Widgets konzipiert:

- **aria-dropeffect** (was passiert beim Loslassen eines Objekts) und
- **aria-grabbed** (Objekt wurde aufgenommen).

Beziehungsattribute: Diese Attribute drücken Beziehungen zwischen Elementen aus:

- **aria-activedescendant** (identifiziert das aktive Unterelement mit Fokus),
- **aria-colcount** (Anzahl der Spalten in einer Tabelle, Grid oder Treegrid),
- **aria-colindex** (Index der aktuellen Spalte),
- **aria-colspan** (Zelle erstreckt sich über mehrere Spalten),
- **aria-controls** (identifiziert das Element, das das aktuelle Element kontrolliert z. B. der Tab das Tabpanel),

- **aria-describedby** (identifiziert das Element, welches das aktuelle beschreibt),
- **aria-details** (identifiziert das Element, das Details über das aktuelle enthält),
- **aria-errormessage** (identifiziert das Objekt, das eine Fehlermeldung für das aktuelle beinhaltet),
- **aria-flowto** (identifiziert das nächste Element, wenn dieses visuell neu angeordnet wurde),
- **aria-labelledby** (identifiziert das Element, das als Label für das aktuelle agiert),
- **aria-owns** (identifiziert das Element, das das aktuelle beinhaltet),
- **aria-posinset** (Index des Elements in einem Set von Listelementen oder Treeitems),
- **aria-rowcount** (Anzahl von Reihen in einer Tabelle, einem Grid oder Treegrid),
- **aria-rowindex** (Index der aktuellen Zeile),
- **aria-rowspan** (Zelle erstreckt sich über mehrere Zeilen) und
- **aria-setsize** (Anzahl der Elemente in einem Set von Listelementen oder Treeitems).

Anwendung von Zuständen und Eigenschaften

Zustände und Eigenschaften – oder kurz Attribute – werden dort verwendet, wo Zuordnungen gemacht werden müssen (um Zusammenhänge darzustellen) oder Dinge sich ändern. Folgendes Beispiel beschreibt einen Tooltip für ein Formularfeld. Der Tooltip wird in einem `<div>`-Element hinterlegt, über JavaScript kann dann gesteuert werden, ob er angezeigt wird (wenn die Maus über dem Formularfeld ist), oder nicht. Dementsprechend muss mit WAI-ARIA-Attributen gekennzeichnet werden, ob der Tooltip sichtbar ist oder nicht (**aria-hidden**). Die Werte der Attribute werden dabei mittels JavaScript mitverändert. Zusätzlich wird noch angegeben, dass der Tooltip das Formularfeld beschreibt (**aria-describedby**) und dass das `<label>`-Element das Formularfeld beschriftet (**aria-labelledby**).

```

1 <div class="text">
2   <label id="tp1-label" for="first">First Name:</label>
3   <input type="text" id="first" name="first" size="20" aria-labelledby="tp1-label"
      aria-describedby="tp1" aria-required="false">
4   <div id="tp1" class="tooltip" role="tooltip" aria-hidden="true">Your first name is
      optional</div>
5 </div>

```

Ist das ARIA-Attribut **true**, wird der Tooltip einfach mittels CSS ausgeblendet:

```

1 div.tooltip[aria-hidden="true"] {
2   display: none;
3 }

```

6.4.3 WAI-ARIA einsetzen

Viele der Rollen und Attribute in WAI-ARIA decken sich mit Elementen aus dem HTML Living Standard. Diese Elemente sollten immer zuerst verwendet werden, dafür muss *kein* WAI-ARIA-Attribut hinzugefügt werden. Folgendes Markup ist daher redundant und *nicht* notwendig:

```
1 <nav role="navigation">...</nav>
```

Ebenso sollen durch die Attribute bestehende Semantiken nicht umdefiniert werden. Anstatt einer `<h2>`-Überschrift die Rolle eines Tabs zu geben, ist es besser, ein allgemeines, bedeutungsloses `<div>`-Element zu nehmen:

```
1 <div role="tab">Header Tab</div>
```

Keine ARIA sind besser als schlechte ARIA

ARIA Rollen, Zustände und Eigenschaften entsprechen in etwa CSS-Anweisungen für unterstützende Technologien. Sie bestimmen, wie diese Technologien Webinhalte für sie wiedergeben. Falsch angewandte ARIA können daher auch verheerende Effekte auf Personen mit Beeinträchtigungen haben. Die WAI hat daher zwei Prinzipien definiert, die vor der Verwendung von ARIA bedacht werden sollten:

1. **Eine Rolle ist ein Versprechen:** Ein Element wie `<div role="button">Place Order</div>` beinhaltet, dass das `<div>`-Element sich wie ein HTML-Button verhält, d. h. auch etwa JavaScript-Code für korrekte Bedienung mit dem Keyboard enthält. Fehlt dies, verlassen sich User*innen darauf, dass dieses Element sich wie ein Button verhält und werden dabei enttäuscht.
2. **ARIA können sowohl verdecken als auch verbessern – dies ist gleichzeitig mächtig und gefährlich:** ARIA kann die ursprüngliche Semantik von Elementen überschreiben oder auch neue Bedeutungen schaffen. Dies ist zwar sehr mächtig, kann bei falscher Anwendung aber auch der Barrierefreiheit schaden.

Der *ARIA Authoring Practice Guide (APG)*²⁰ bietet praktische Beispiele zur Implementierung von gängigen Webkomponenten. Hier finden sich auch die zwei soeben beschriebenen Prinzipien²¹. Ebenso finden sich Informationen zur Umsetzung von ARIA im Dokument *Using ARIA*²².

6.5 Unterstützende Technologien

Damit beeinträchtigte Personen auf Webinhalte zugreifen können, benötigen sie mitunter Technologien, die etwa einen Bildschirm ersetzen bzw. bei der Betrachtung unterstützend eingreifen. Gebräuchliche Werkzeuge sind Screenreader, die den Bildschirminhalt vorlesen oder über Braillezeilen in Brailleschrift ausgeben.

Um Screenreader selbst einfach zu testen ist vor allem *NVDA*²³ (Non Visual Desktop Access) empfehlenswert, da es sich um einen der wenigen Open Source Screenreader handelt. Andere, bekannte Produkte wie etwa der Marktführer *JAWS*²⁴ sind kostenpflichtig.

Weiters haben die bekannten Betriebssysteme meist Bildschirmlesetechnologien bereits integriert. So existiert für macOS etwa *VoiceOver*²⁵, Windows 11 haben den *Micro-*

²⁰<https://www.w3.org/WAI/ARIA/apg/>

²¹<https://www.w3.org/WAI/ARIA/apg/practices/read-me-first/>

²²<https://www.w3.org/TR/using-aria/>

²³<https://www.nvaccess.org/>

²⁴<https://www.freedomscientific.com/Products/software/JAWS/>

²⁵<https://support.apple.com/de-at/guide/voiceover-guide/welcome/web>

soft Narrator (auf Deutsch einfach „Sprachausgabe“ genannt)²⁶ integriert. Diese Tools lassen sich ohne weitere Installation nützen und erlauben ein Testen und eigenständiges Erfahren von Screenreader-Technologien.

6.5.1 Testen mit unterstützenden Technologien

Um einen Eindruck zu bekommen, wie Personen, die auf Screenreader angewiesen sind, die eigene Webseite wahrnehmen, empfiehlt es sich, eines der soeben genannten Werkzeuge zu installieren und die eigene Seite damit zu betrachten. So lassen sich sehr schnell Probleme mit unzureichenden Alternativtexten für Bilder, problematische Lesereihenfolge, Probleme mit Formularen oder weitere Accessibility-Aspekte herausfinden.

Einige Überlegungen und einen guten Startpunkt mit weiteren Links zu diversen Technologien findet sich unter https://webaim.org/articles/screenreader_testing/.

6.6 Weiterführende Literatur

Die Literatur zu Web Accessibility ist spärlich. Es existieren meist Bücher die zehn Jahre oder älter sind. Mit [2] gibt es jedoch ein sehr neues Werk, das Barrierefreiheit im Web auf einfache Art erklärt und dabei auch die WCAG 2.2 bereits berücksichtigt.

Aktuell ist ebenfalls [1], in dem die Autorin zunächst verschiedene Probleme der Barrierefreiheit erläutert und dann mit Hilfe von Webtechnologien wie ARIA oder CSS Lösungen für diese Probleme anbietet. Auch in [4] finden sich die Themen WCAG und WAI-ARIA zusammen mit der rechtlichen Dimension.

Wer ein praxisorientiertes Buch mit konkreten Vorschlägen zur Gestaltung von Komponenten sucht, kann auf [5] zurückgreifen. Das Buch behandelt weniger die Grundlagen sondern liefert konkrete Codebeispiele. Es wurde von einem Österreicher geschrieben.

Darüber hinaus sind die Standard-Dokumente zu WCAG [16] und WAI-ARIA [9] die beste Ressource für Informationen.

²⁶<https://support.microsoft.com/de-de/windows/vollst%C3%A4ndige-anleitung-f%C3%BCr-die-sprachausgabe-e4397a0d-ef4f-b386-d8ae-c172f109bdb1>

Quellenverzeichnis

- [1] Ashley Firth. *Practical Web Accessibility. A Comprehensive Guide to Digital Inclusion*. 2. Aufl. London, Vereinigtes Königreich: Apress, 18. Apr. 2024 (siehe S. 6-22).
- [2] Dimitris Georgakas. *A11Y Unraveled. Become a Web Accessibility Ninja*. New York, USA: Apress, 24. März 2023 (siehe S. 6-22).
- [3] International Organization for Standardization. *W3C Accessibility Guidelines (WCAG) 3.0*. W3C Working Draft. Sep. 2025. URL: <https://www.iso.org/standard/91029.html> (siehe S. 6-6).
- [4] Maria Korneeva. *Barrierefreie Webentwicklung. Von den Grundlagen über die rechtlichen Aspekte bis zur praktischen Umsetzung*. O'Reilly, 30. Mai 2025 (siehe S. 6-22).
- [5] Manuel Matuzović. *Web Accessibility Cookbook. Creating Inclusive Experiences*. O'Reilly, 23. Juli 2024 (siehe S. 6-22).
- [6] MDN Contributors. *An overview of accessible web applications and widgets*. 7. Juni 2025. URL: https://developer.mozilla.org/en-US/docs/Web/Accessibility/Guides/Accessible_web_applications_and_widgets (besucht am 28. 11. 2025) (siehe S. 6-18).
- [7] W3C Web Accessibility Initiative. *Essential Components of Web Accessibility*. Hrsg. von Shawn Lawton Henry und Michael Duffy. 14. März 2024. URL: <https://www.w3.org/WAI/fundamentals/components/> (besucht am 28. 11. 2025) (siehe S. 6-4).
- [8] W3C Web Accessibility Initiative. *Introduction to Web Accessibility*. Hrsg. von Shawn Lawton Henry. 7. März 2024. URL: <https://www.w3.org/WAI/fundamentals/accessibility-intro/> (besucht am 28. 11. 2025) (siehe S. 6-1).
- [9] World Wide Web Consortium. *Accessible Rich Internet Applications (WAI-ARIA) 1.2*. W3C Recommendation. 6. Juni 2023. URL: <https://www.w3.org/TR/wai-aria-1.2/> (siehe S. 6-14, 6-18, 6-22).
- [10] World Wide Web Consortium. *Accessible Rich Internet Applications (WAI-ARIA) 1.3*. W3C First Public Working Draft. 23. Jan. 2024. URL: <https://www.w3.org/TR/wai-aria-1.3/> (siehe S. 6-14).
- [11] World Wide Web Consortium. *Authoring Tool Accessibility Guidelines (ATAG) 2.0*. W3C Recommendation. 24. Sep. 2015. URL: <https://www.w3.org/TR/ATAG20/> (siehe S. 6-5).

- [12] World Wide Web Consortium. *Synchronization Accessibility User Requirements*. W3C Group Note. 28. Juni 2023. URL: <https://www.w3.org/TR/saur/> (siehe S. 6-3).
- [13] World Wide Web Consortium. *User Agent Accessibility Guidelines (UAAG) 2.0*. W3C Working Group Note. 15. Dez. 2015. URL: <https://www.w3.org/TR/UAAG20/> (siehe S. 6-5).
- [14] World Wide Web Consortium. *W3C Accessibility Guidelines (WCAG) 3.0*. W3C Working Draft. 4. Sep. 2025. URL: <https://www.w3.org/TR/wcag-3.0/> (siehe S. 6-6).
- [15] World Wide Web Consortium. *Web Accessibility Initiative*. 2025. URL: <https://www.w3.org/WAI/> (besucht am 28.11.2025) (siehe S. 6-2).
- [16] World Wide Web Consortium. *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. 24. Dez. 2024. URL: <https://www.w3.org/TR/WCAG22/> (siehe S. 6-6, 6-7, 6-22).
- [17] World Wide Web Consortium. *Website Accessibility Conformance Evaluation Methodology (WCAG-EM) 1.0*. W3C Working Group Note. 10. Juli 2014. URL: <https://www.w3.org/TR/WCAG-EM/> (siehe S. 6-11, 6-13).
- [18] World Wide Web Consortium. *XR Accessibility User Requirements*. W3C Working Group Note. 25. Aug. 2021. URL: <https://www.w3.org/TR/xaur/> (siehe S. 6-3).